COMP0005 – ALGORITHMS

Week 10 – 25 March 2025

Prof Licia Capra

Dept of Computer Science

OUTLINE

- Week Preview
- Exercises on Strings
- Assessment Info

WEEK PREVIEW

- Topic 5 – Strings
  - 5.3 Substring search
  - 5.4 Compression
- Self-practice
- TA Lab – Week 10
- *No Muddiest Point (Weeks 9 & 10 - Strings)*

- The End – Recap lecture

OUTLINE

- Week Preview
- Exercises on Strings
  - String Sort – LSD
  - String Search – 3way Trie
  - Substring Search – KMP
  - Compression – Huffman Code Construction
- Assessment Info

EXERCISE – STRING SORT

- 2021-22 LSA Exam – Question 4(a)

Question 4 (Strings)

(a) The array on the left (labelled "input") represents an original input of 10 strings to be sorted. The array on the right (labelled "sorted") contains these strings in sorted order.

input	sorted
crow	calf
swan	crab
wolf	crow
slug	lamb
calf	lion
lion	slug
sole	sole
wasp	swan
crab	wasp
lamb	wolf

Fully illustrate the steps taken by the LSD radix sort algorithm to transform the "input" array into the "sorted" array.

[9 marks]

Least-significant-digit (LSD) first radix sort

Assumption: Keys all have the same (small) length *D*

Basic idea:

- Consider each character in turn, from right to left
- At each pass, use key-indexed counting on d^{th} character to sort (stably)

EXERCISE – STRING SORT

- 2021-22 LSA Exam – Question 4(a)

Question 4 (Strings)

(a) The array on the left (labelled "input") represents an original input of 10 strings to be sorted. The array on the right (labelled "sorted") contains these strings in sorted order.

input	sorted
crow	calf
swan	crab
wolf	crow
slug	lamb
calf	lion
lion	slug
sole	sole
wasp	swan
crab	wasp
lamb	wolf

Fully illustrate the steps taken by the LSD radix sort algorithm to transform the "input" array into the "sorted" array.

[9 marks]

```
def LSDsort(a, D):
    N = len(a)
    for d in range (D-1, 0, -1):
        count = [0 for _ in range(R+1)]

        # key-indexed counting
        for i in range(0, N):
            count[a[i][d]+1]++
        for r in range (0, R):
            count[r+1] = count[r+1] + count[r]
        for i in range (0, N):
            aux[count[a[i][d]]] = a[i]
            count[a[i][d]]++
        for i in range(0, N):
            a[i] = aux[i]
```

EXERCISE – STRING SORT

- 2021-22 LSA Exam – Question 4(a)

input

crow
swan
wolf
slug
calf
lion
sole
wasp
crab
lamb

```
def LSDsort(a, D):
    N = len(a)
    for d in range (D-1, 0, -1):
        count = [0 for _ in range(R+1)]

        # key-indexed counting
        for i in range(0, N):
            count[a[i][d]+1]++
        for r in range (0, R):
            count[r+1] = count[r+1] + count[r]
        for i in range (0, N):
            aux[count[a[i][d]]] = a[i]
            count[a[i][d]]++
        for i in range(0, N):
            a[i] = aux[i]
```

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
count	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

EXERCISE – STRING SORT

- 2021-22 LSA Exam – Question 4(a)

input

crow
swan
wolf
slug
calf
lion
sole
wasp
crab
lamb

```
def LSDsort(a, D):
    N = len(a)
    for d in range (D-1, 0, -1):
        count = [0 for _ in range(R+1)]

        # key-indexed counting
        for i in range(0, N):
            count[a[i][d]+1]++
        for r in range (0, R):
            count[r+1] = count[r+1] + count[r]
        for i in range (0, N):
            aux[count[a[i][d]]] = a[i]
            count[a[i][d]]++
        for i in range(0, N):
            a[i] = aux[i]
```

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
count	0	0	2	0	0	1	2	1	0	0	0	0	0	0	2	0	1	0	0	0	0	0	0	1	0	0	0

EXERCISE – STRING SORT

- 2021-22 LSA Exam – Question 4(a)

input

crow
swan
wolf
slug
calf
lion
sole
wasp
crab
lamb

```
def LSDsort(a, D):
    N = len(a)
    for d in range (D-1, 0, -1):
        count = [0 for _ in range(R+1)]

        # key-indexed counting
        for i in range(0, N):
            count[a[i][d]+1]++
        for r in range (0, R):
            count[r+1] = count[r+1] + count[r]
        for i in range (0, N):
            aux[count[a[i][d]]] = a[i]
            count[a[i][d]]++
        for i in range(0, N):
            a[i] = aux[i]
```

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
count	0	0	2	2	2	3	5	6	6	6	6	6	6	6	8	8	9	9	9	9	9	9	9	10	10	10	10

EXERCISE – STRING SORT

- 2021-22 LSA Exam – Question 4(a)

input

crow
swan
wolf
slug
calf
lion
sole
wasp
crab
lamb

```
def LSDsort(a, D):
    N = len(a)
    for d in range (D-1, 0, -1):
        count = [0 for _ in range(R+1)]

        # key-indexed counting
        for i in range(0, N):
            count[a[i][d]+1]++
        for r in range (0, R):
            count[r+1] = count[r+1] + count[r]
        for i in range (0, N):
            aux[count[a[i][d]]] = a[i]
            count[a[i][d]]++
        for i in range(0, N):
            a[i] = aux[i]
```

aux

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
count	0	0	2	2	2	3	5	6	6	6	6	6	6	6	8	8	9	9	9	9	9	9	9	10	10	10	10

EXERCISE – STRING SORT

- 2021-22 LSA Exam – Question 4(a)

input

crow
swan
wolf
slug
calf
lion
sole
wasp
crab
lamb

```
def LSDsort(a, D):
    N = len(a)
    for d in range (D-1, 0, -1):
        count = [0 for _ in range(R+1)]

        # key-indexed counting
        for i in range(0, N):
            count[a[i][d]+1]++
        for r in range (0, R):
            count[r+1] = count[r+1] + count[r]
        for i in range (0, N):
            aux[count[a[i][d]]] = a[i]
            count[a[i][d]]++
        for i in range(0, N):
            a[i] = aux[i]
```

aux

0	crab
1	lamb
2	sole
3	wolf
4	calf
5	slug
6	swan
7	lion
8	wasp
9	crow

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
count	0	2	2	2	3	5	6	6	6	6	6	6	6	8	8	9	9	9	9	9	9	9	10	10	10	10	10

EXERCISE – STRING SORT

- 2021-22 LSA Exam – Question 4(a)

input

crow
swan
wolf
slug
calf
lion
sole
wasp
crab
lamb

```
def LSDsort(a, D):
    N = len(a)
    for d in range (D-1, 0, -1):
        count = [0 for _ in range(R+1)]

        # key-indexed counting
        for i in range(0, N):
            count[a[i][d]+1]++
        for r in range (0, R):
            count[r+1] = count[r+1] + count[r]
        for i in range (0, N):
            aux[count[a[i][d]]] = a[i]
            count[a[i][d]]++
        for i in range(0, N):
            a[i] = aux[i]
```

	aux
0	crab
1	lamb
2	sole
3	wolf
4	calf
5	slug
6	swan
7	lion
8	wasp
9	crow

	a
0	crab
1	lamb
2	sole
3	wolf
4	calf
5	slug
6	swan
7	lion
8	wasp
9	crow

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
count	0	2	2	2	3	5	6	6	6	6	6	6	6	8	8	9	9	9	9	9	9	9	10	10	10	10	10

EXERCISE – STRING SORT

- 2021-22 LSA Exam – Question 4(a) [SOLUTION]

crow	crab	crab	calf	calf
swan	lamb	swan	lamb	crab
wolf	sole	sole	wasp	crow
slug	wolf	wolf	lion	lamb
calf	calf	calf	slug	lion
lion	slug	lamb	sole	slug
Sole	swan	lion	wolf	sole
wasp	lion	crow	crab	swan
crab	wasp	wasp	crow	wasp
lamb	crow	slug	swan	wolf

OUTLINE

- Week Preview
- Exercises on Strings
  - String Sort – LSD
  - String Search – 3way Trie
  - Substring Search – KMP
  - Compression – Huffman Code Construction
- Assessment Info

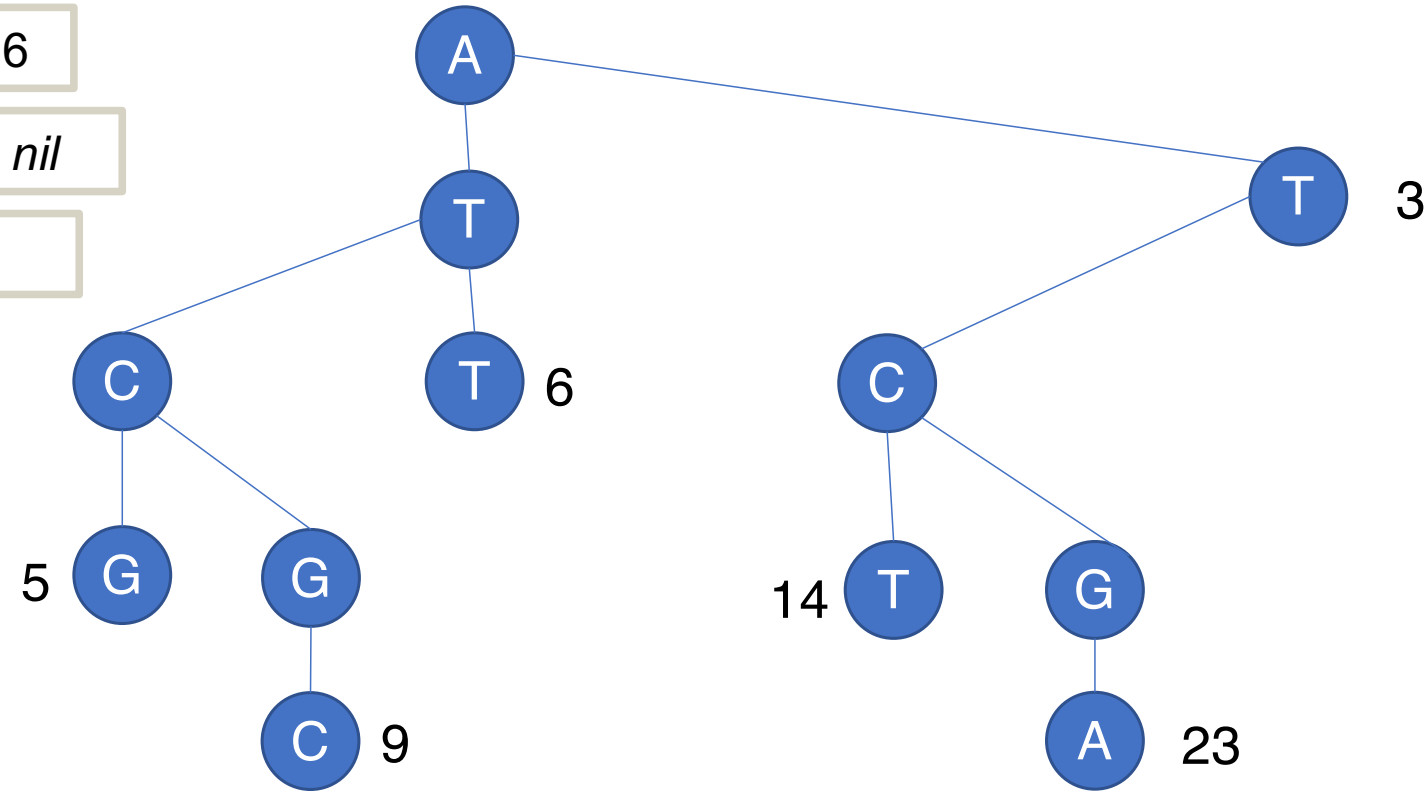
EXERCISE – STRING SEARCH

- Given the ternary (3-way) trie below, what sequence of inserts could have generated it?

Search: ATT → return 6

Search: ACT → return *nil*

Search: CGA → … ?



EXERCISE – STRING SEARCH

- Given the ternary (3-way) trie below, what sequence of inserts could have generated it?

1) ATT ACG T CT AGC GA

2) ATT T CT ACG GA AGC

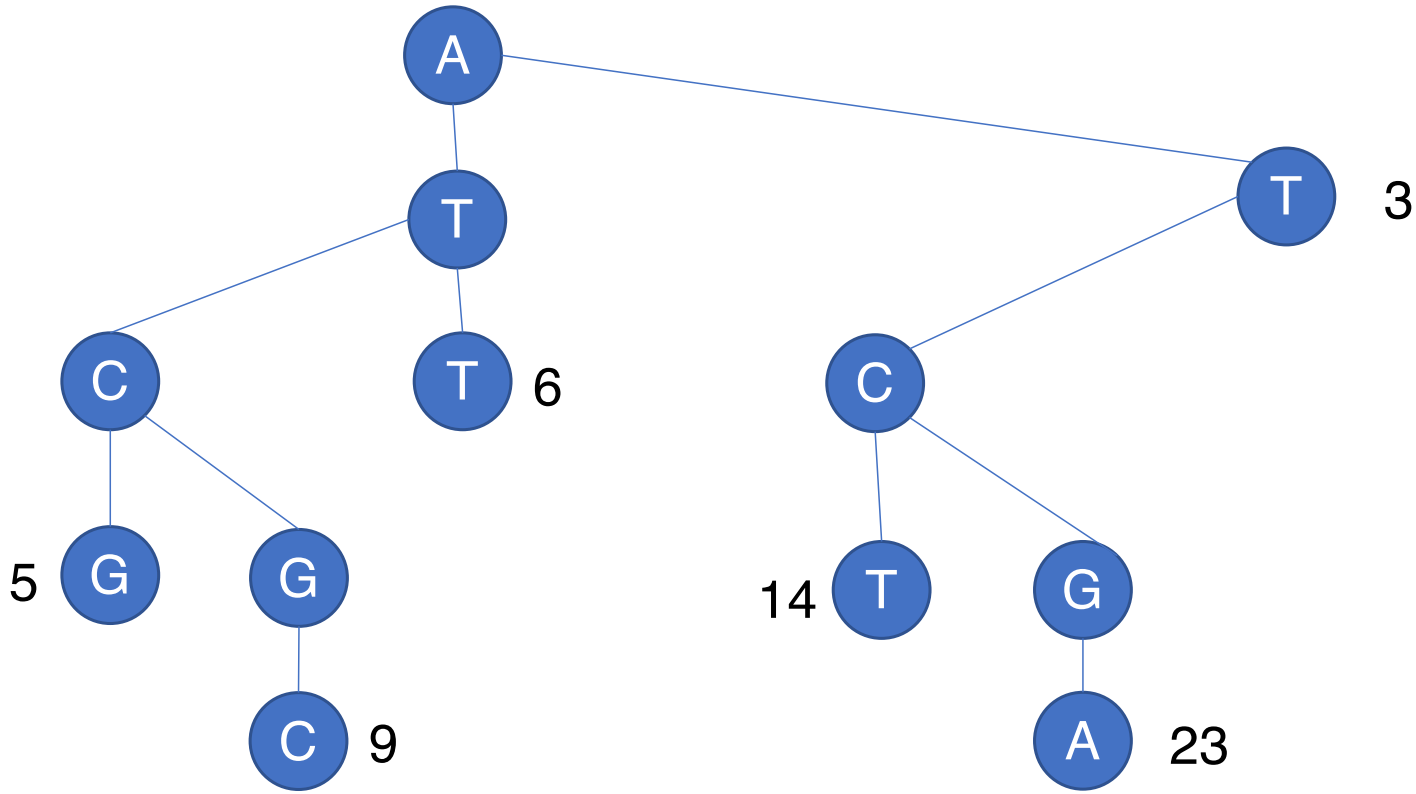
3) ATT T GA ACG CT AGC

4) ATT ACG AGC T CT GA

5) ATT ACG T CT GA AGC

6) ATT T AGC ACG CT GA

7) ATT T AGC GA ACG CT



EXERCISE – STRING SEARCH

- Given the ternary (3-way) trie below, what sequence of inserts could have generated it? [SOLUTION]

1) ATT ACG T CT AGC GA

2) ATT T CT ACG GA AGC

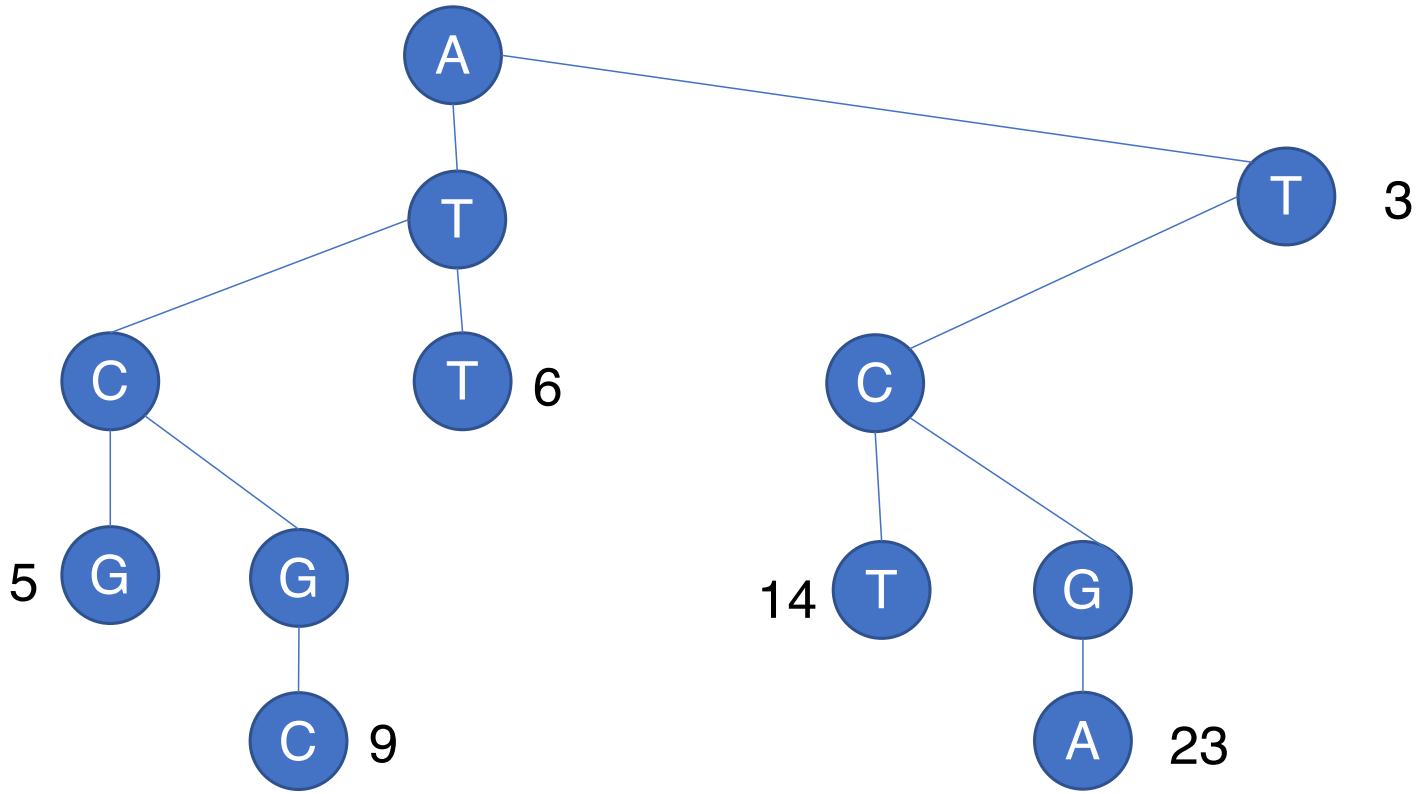
~~3) ATT T GA ACG CT AGC~~

4) ATT ACG AGC T CT GA

5) ATT ACG T CT GA AGC

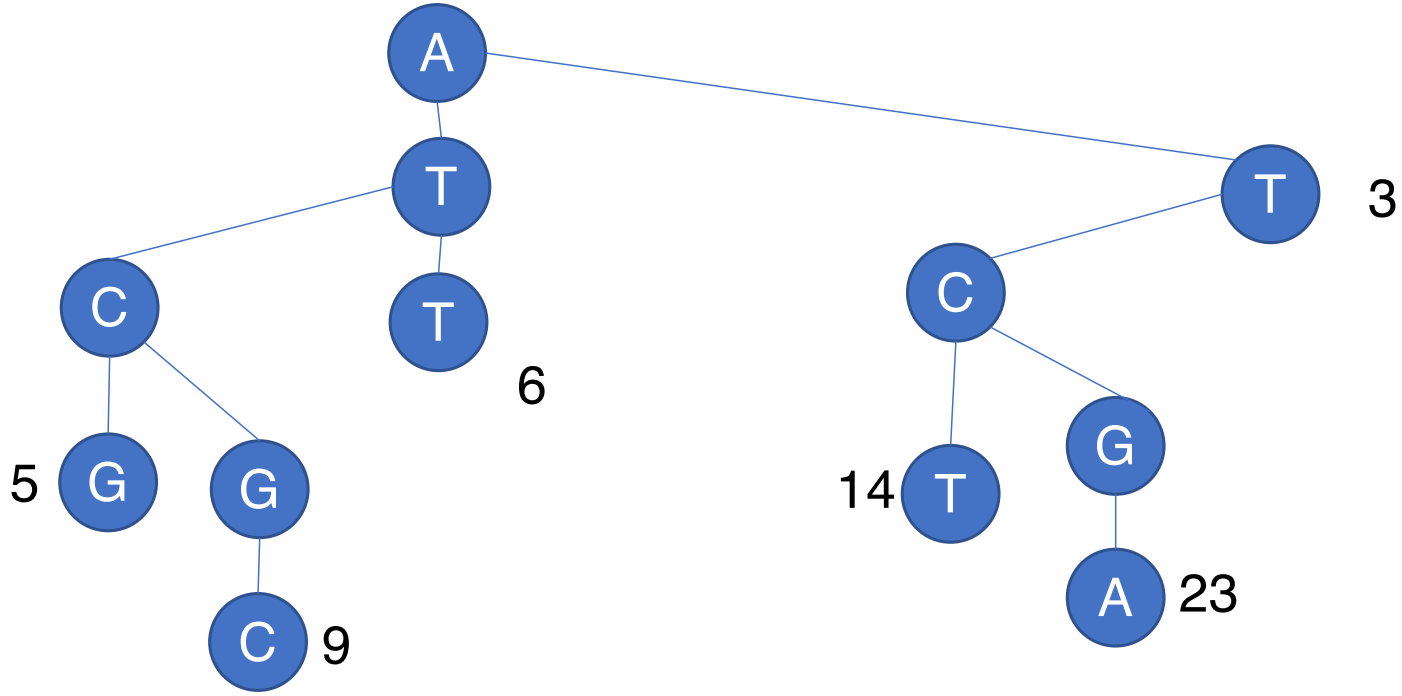
~~6) ATT T AGC ACG CT GA~~

~~7) ATT T AGC GA ACG CT~~



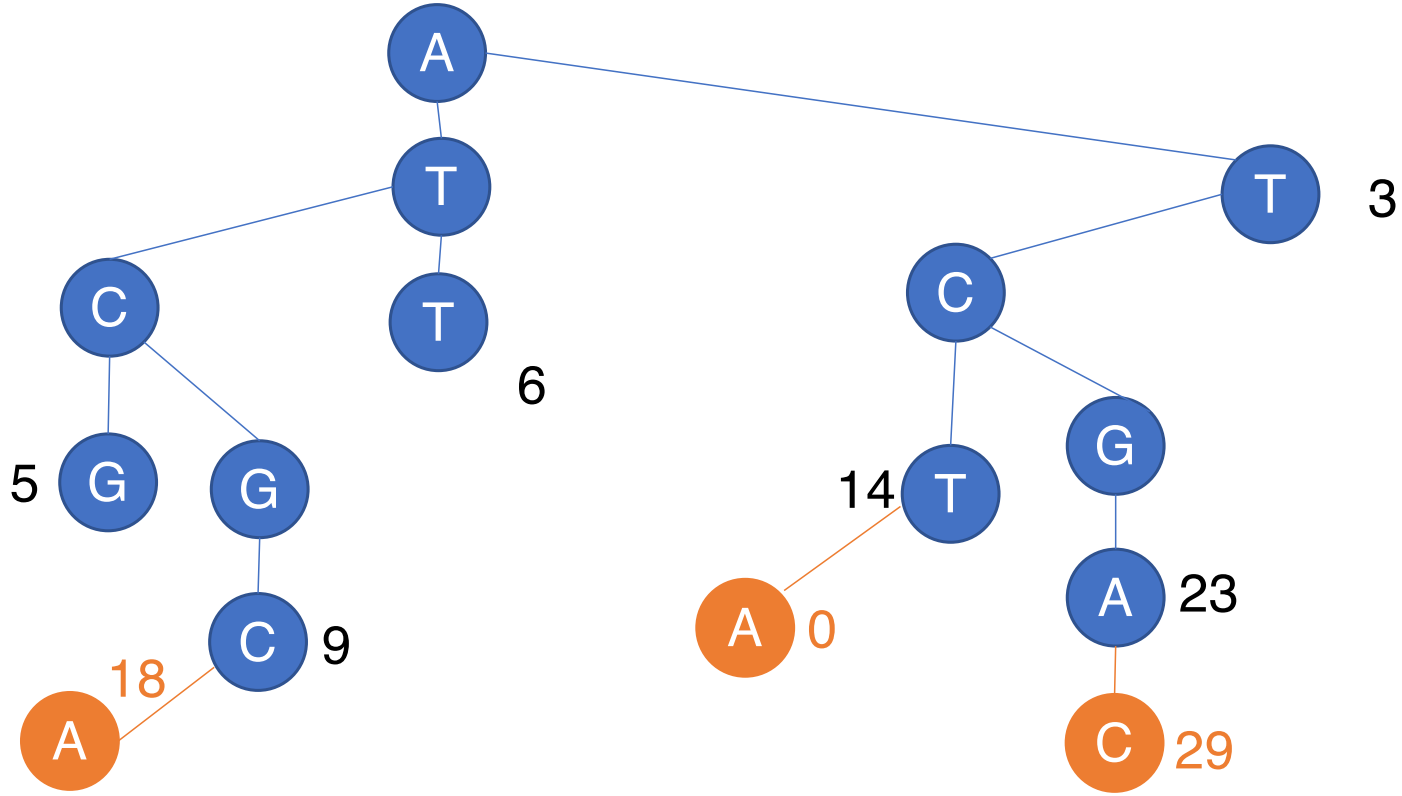
EXERCISE – STRING SEARCH

- Update the 3-way trie below, after inserting (CA, 0), (AGA, 18), and (GAC, 29).



EXERCISE – STRING SEARCH

- Update the 3-way trie below, after inserting (CA, 0), (AGA, 18), and (GAC, 29). [SOLUTION]



OUTLINE

- Week Preview
- Exercises on Strings
  - String Sort – LSD
  - String Search – 3way Trie
  - Substring Search – KMP
  - Compression – Huffman Code Construction
- Assessment Info

EXERCISE – SUBSTRING SEARCH

- 2021-22 Main Exam – Question 4(a)

Question 4 (Strings)

(a) Given the alphabet {A, B, C} and a pattern *p* of length 7:

ABAAABC

Construct the DFA used by the Knuth-Morris-Pratt (KMP) algorithm to search for occurrences of *p* within a main text string *s*.

[10 marks]

EXERCISE – SUBSTRING SEARCH

- How to build the DFA from pattern
  1) Init. Create one state per character in the pattern, plus a final state
  2) Match transitions. If in state *j* and next char `c==pattern[j]`, go to state *j+1*
  3) Mismatch transitions.
     - If in state *j* and next char `c != pattern[j]`, then the last *j-1* characters in input are `pattern[1..j-1]`, followed by `c`.
     - To fill in `DFA[c][j]`, simulate having `pattern[1..j-1]` in input to the DFA, then take transition `c`.

As if re-processing the already seen portion of the input

Move forward one (unseen) character

EXERCISE – SUBSTRING SEARCH

- How to build the DFA from pattern
  1) Init. Create one state per character in the pattern, plus a final state
  2) Match transitions. If in state *j* and next char `c==pattern[j]`, go to state *j+1*
  3) Mismatch transitions.
     - If in state *j* and next char `c != pattern[j]`, then the last *j-1* characters in input are `pattern[1..j-1]`, followed by `c`.
     - To fill in `DFA[c][j]`, simulate having `pattern[1..j-1]` in input to the DFA, then take transition `c`.

Move forward one (unseen) character

As if re-processing the already seen portion of the input

	0	1	2	3	4	5	6	7
	A	B	A	A	A	B	C	
A								
B								
C								

EXERCISE – SUBSTRING SEARCH

- How to build the DFA from pattern
  1) Init. Create one state per character in the pattern, plus a final state
  2) Match transitions. If in state *j* and next char `c==pattern[j]`, go to state *j+1*
  3) Mismatch transitions.
     - If in state *j* and next char `c != pattern[j]`, then the last *j-1* characters in input are `pattern[1..j-1]`, followed by `c`.
     - To fill in `DFA[c][j]`, simulate having `pattern[1..j-1]` in input to the DFA, then take transition `c`.

Move forward one (unseen) character

As if re-processing the already seen portion of the input

	0	1	2	3	4	5	6	7
	A	B	A	A	A	B	C	
A	1		3	4	5			
B		2				6		
C							7	

EXERCISE – SUBSTRING SEARCH

- How to build the DFA from pattern
  1) Init. Create one state per character in the pattern, plus a final state
  2) Match transitions. If in state *j* and next char `c==pattern[j]`, go to state *j+1*
  3) Mismatch transitions.
     - If in state *j* and next char `c != pattern[j]`, then the last *j-1* characters in input are `pattern[1..j-1]`, followed by `c`.
     - To fill in `DFA[c][j]`, simulate having `pattern[1..j-1]` in input to the DFA, then take transition `c`.

Move forward one (unseen) character

As if re-processing the already seen portion of the input

	0	1	2	3	4	5	6	7
	A	B	A	A	A	B	C	
A	1	1	3	4	5	1	3	
B	0	2	0	2	2	6	0	
C	0	0	0	0	0	0	7	

EXERCISE – SUBSTRING SEARCH

- 2021-22 Main Exam – Question 4(a) [SOLUTION]

Question 4 (Strings)

(a) Given the alphabet {A, B, C} and a pattern *p* of length 7:

ABAAABC

Construct the DFA used by the Knuth-Morris-Pratt (KMP) algorithm to search for occurrences of *p* within a main text string *s*.

[10 marks]

	0	**1**	**2**	**3**	**4**	**5**	**6**	**7**
A	1	1	3	4	5	1	3	
B	0	2	0	2	2	6	0	
C	0	0	0	0	0	0	7	

OUTLINE

- Week Preview
- Exercises on Strings
  - String Sort – LSD
  - String Search – 3way Trie
  - Substring Search – KMP
  - Compression – Huffman Code Construction
- Assessment Info

EXERCISE – COMPRESSION

Given input message M:

"`compression`"

with characters taken from alphabet [`a-z`], use Huffman compression to determine the optimal prefix-free codewords for characters in M.

EXERCISE – COMPRESSION

M = "`compression`"

Huffman Algorithm – Code Construction

- count frequency `freq[i]` for each char `i` in input
- create a trie node for each char `i` and assign weight = `freq[i]`
- repeat until a single trie is formed:
  - select the two tries with minimum weights `freq[i]` and `freq[j]`
  - merge into a single trie with weight `freq[i] + freq[j]`

char	c	e	i	m	n	o	p	r	s
freq[]	**1**	**1**	**1**	**1**	**1**	**2**	**1**	**1**	**2**

EXERCISE – COMPRESSION

M = "`compression`" [SOLUTION]

Huffman Algorithm – Code Construction

- count frequency `freq[i]` for each char `i` in input
- create a trie node for each char `i` and assign weight = `freq[i]`
- repeat until a single trie is formed:
  - select the two tries with minimum weights `freq[i]` and `freq[j]`
  - merge into a single trie with weight `freq[i] + freq[j]`

```
def buildTrie():
    pq = new MinPQ()
    for i in range (0, R):
        if (freq[i] > 0):
            newNode = Node(i, freq[i], null, null)
            pq.insert(newNode)

    while (pq.size() > 1):
        x = pq.delMin()
        y = pq.delMin()
        parent = Node('', x.freq + y.freq, x, y)
        pq.insert(parent)

    return pq.delMin()
```

char	c	e	i	m	n	o	p	r	s
freq[]	**1**	**1**	**1**	**1**	**1**	**2**	**1**	**1**	**2**
encoding	1110	1111	000	001	010	101	011	100	110

OUTLINE

- Week Preview
- Exercises on Strings
- Assessment Info

THE END ☺

- Coursework
  - *Deadline passed (24/03, SoRA 31/03)*
  - Marks to be returned by 25/04
- Exam
  - Wednesday 07/05, 14:30-16:30 (2 hours)
  - Exam Rubric:
    - There are FOUR questions in total
    - Answer ALL questions
    - A maximum of 100 marks are available
  - No calculators permitted
  - Past exam papers available via UCL Library